

Lista de exercícios 01: primeiros passos com containers

Encontro 01: containerização e Docker

Como usar esta lista

- Os dez exercícios são progressivos: cada um assume que o anterior foi executado.
- Todos os comandos rodam em um terminal Linux com o Docker instalado e o serviço ativo.
- Nenhum exercício exige escrever Dockerfile ou usar volumes, que serão tratados no próximo encontro.
- Registre as saídas relevantes de cada exercício. A entrega é o registro do que você observou, e não apenas o comando executado.
- Ao encontrar uma mensagem de erro, consulte a tabela de erros comuns no final desta lista antes de pedir ajuda.

Exercício 1: Preparar o ambiente e ler a instalação

Objetivo

Confirmar que o Docker está instalado e funcionando, e identificar como o engine está configurado na sua máquina.

Enunciado

1. Verifique a versão do cliente e do engine com `docker version`. Observe que são dois componentes distintos.
2. Execute `docker run hello-world` e leia a mensagem completa que aparece na tela.
3. Execute `docker info` e localize as linhas Cgroup Driver, Cgroup Version e Storage Driver.
4. Anote os três valores encontrados. Eles serão usados nos exercícios 8 e 9.

Comandos de apoio

```
docker version
docker run hello-world
docker info | grep -i -E 'cgroup|storage driver'
```

Como verificar

A mensagem do `hello-world` aparece na tela e você consegue informar a versão de cgroup e o storage driver da sua máquina.

Para pensar

A mensagem do `hello-world` descreve quatro passos executados pelo Docker. Quais deles envolvem a rede e quais acontecem apenas na sua máquina?

Exercício 2: Primeiro container interativo e os namespaces

Objetivo

Perceber, na prática, que o container tem uma visão própria do sistema.

Enunciado

1. Suba um container interativo com a imagem alpine.
2. Dentro do container, execute `hostname`, `ps aux`, `ip addr` e `ls /`.
3. Abra um segundo terminal, no host, e execute os mesmos comandos.
4. Registre três diferenças entre as duas saídas e associe cada uma a um namespace (`uts`, `pid`, `net` ou `mnt`).

Comandos de apoio

```
docker run -it --rm --name lab alpine sh
# dentro do container
hostname ; ps aux ; ip addr ; ls /
# no host, em outro terminal
hostname ; ps aux | head ; ip addr
```

Como verificar

Você consegue apontar três diferenças e dizer qual namespace explica cada uma.

Para pensar

Dentro do container o comando `ps` mostra pouquíssimos processos. Eles deixaram de existir, ou apenas deixaram de ser visíveis?

Exercício 3: Ciclo de vida: run, start e stop

Objetivo

Distinguir a criação de um container novo da retomada de um container existente.

Enunciado

1. Execute três vezes o comando `docker run alpine echo "ola"`.
2. Liste os containers em execução com `docker ps` e depois todos com `docker ps -a`. Compare as duas listas.
3. Escolha um dos containers parados e reinicie com `docker start -a <ID ou nome>`.
4. Crie um container nomeado com `--name teste` e tente criar outro com o mesmo nome. Leia a mensagem de erro.

Comandos de apoio

```
docker run alpine echo "ola"
docker ps
docker ps -a
docker start -a <ID>
docker run --name teste alpine echo "ola"
```

Como verificar

`docker ps -a` mostra três containers distintos criados pelos três `run`, e o `start` reaproveita um deles em vez de criar um quarto.

Para pensar

Por que executar `docker run` duas vezes não reaproveita o primeiro container? O que exatamente o `run` cria a partir da imagem?

Exercício 4: Attached, detached e leitura de logs

Objetivo

Escolher o modo de execução adequado e acompanhar a saída de um serviço em segundo plano.

Enunciado

1. Suba um nginx em modo attached, sem a flag -d, e observe que o terminal fica ocupado. Interrompa com Ctrl+C.
2. Suba o mesmo nginx em modo detached, com -d e --name web.
3. Leia a saída do serviço com docker logs web e depois acompanhe em tempo real com docker logs -f web.
4. Confirme com docker ps que o serviço continua ativo após você sair do acompanhamento.

Comandos de apoio

```
docker run --name web1 nginx:alpine
docker run -d --name web nginx:alpine
docker logs web
docker logs -f web
```

Como verificar

O serviço permanece em execução com o terminal livre, e você consegue ler os registros sem interromper o container.

Para pensar

Sair do docker logs -f encerra o container? E sair de um container iniciado com docker run -it, encerra?

Exercício 5: Publicando portas

Objetivo

Conectar a rede do container à rede do host e entender a ordem da publicação.

Enunciado

1. Remova o container do exercício anterior e suba um nginx publicando a porta 8080 do host na porta 80 do container.
2. Acesse `http://localhost:8080` pelo navegador ou pelo curl.
3. Confirme o mapeamento com `docker port web`.
4. Sem remover o primeiro, tente subir um segundo container publicando a mesma porta 8080. Leia o erro.
5. Suba esse segundo container na porta 8081 e confirme que os dois respondem.

Comandos de apoio

```
docker rm -f web
docker run -d --name web -p 8080:80 nginx:alpine
curl http://localhost:8080
docker port web
docker run -d --name web2 -p 8081:80 nginx:alpine
```

Como verificar

Os dois serviços respondem em portas diferentes do host, e você explica o que significa cada lado da notação 8080:80.

Para pensar

Sem a publicação de porta, o nginx dentro do container está executando? Ele está apenas inacessível, ou está parado?

Exercício 6: Executando comandos dentro de um container em execução

Objetivo

Trabalhar dentro de um container ativo e observar o destino da camada de escrita.

Enunciado

1. Entre no container web com docker exec em modo interativo.
2. Crie um arquivo dentro do container, por exemplo em /tmp/ anotacao.txt, e saia.
3. Entre novamente e confirme que o arquivo continua lá.
4. Pare e reinicie o container com stop e start, e verifique se o arquivo sobreviveu.
5. Remova o container com docker rm -f, crie outro a partir da mesma imagem e procure o arquivo.

Comandos de apoio

```
docker exec -it web sh
echo "teste" > /tmp/anotacao.txt ; cat /tmp/anotacao.txt ; exit
docker stop web && docker start web
docker exec web cat /tmp/anotacao.txt
docker rm -f web && docker run -d --name web -p 8080:80 nginx:alpine
```

Como verificar

O arquivo sobrevive ao stop e ao start, e desaparece após a remoção do container.

Para pensar

*O que exatamente foi perdido quando o container foi removido: a imagem, a camada de escrita ou as duas?
Que recurso do Docker resolveria essa perda?*

Exercício 7: Imagens, tags e camadas

Objetivo

Comparar imagens da mesma família e observar a estrutura em camadas.

Enunciado

1. Baixe as imagens python:3.12-alpine e python:3.12-slim.
2. Liste as imagens locais e compare os tamanhos.
3. Inspecione o histórico de camadas de cada uma com docker image history.
4. Baixe a imagem python:3.12-slim novamente e observe o tempo e a mensagem retornada.
5. Registre o número de camadas e o tamanho de cada imagem em uma tabela simples.

Comandos de apoio

```
docker pull python:3.12-alpine
docker pull python:3.12-slim
docker images
docker image history python:3.12-slim
docker image inspect python:3.12-alpine --format '{{len .RootFS.Layers}} camadas'
```

Como verificar

Você apresenta o tamanho e a quantidade de camadas das duas imagens e explica a diferença entre as tags alpine e slim.

Para pensar

Se duas imagens diferentes compartilham camadas iniciais, o espaço em disco ocupado é a soma dos dois tamanhos exibidos pelo docker images? Por quê?

Exercício 8: Limitando memória: cgroups na prática

Objetivo

Aplicar um limite de memória e observar o comportamento do kernel quando ele é atingido.

Enunciado

1. Suba um container com limite de 64 MB de memória e sem swap adicional, executando um programa que tenta alocar 300 MB.
2. Observe o encerramento do container e consulte o código de saída e o indicador de OOM.
3. Repita a mesma tarefa com um limite de 512 MB e compare o resultado.
4. Com um container ativo e limitado, leia o valor de `memory.max` direto do sistema de arquivos de cgroups do host.
5. Use o caminho compatível com o cgroup driver anotado no exercício 1.

Comandos de apoio

```
docker run --name estouro --memory=64m --memory-swap=64m python:3.12-alpine \
  python -c 'a = bytearray(300 * 1024 * 1024)'
docker inspect estouro --format '{{.State.ExitCode}} {{.State.OOMKilled}}'
docker run --rm --memory=512m python:3.12-alpine \
  python -c 'a = bytearray(300 * 1024 * 1024); print("alocou")'
docker run -d --name limitado --memory=128m nginx:alpine
docker inspect -f '{{.Id}}' limitado
# driver systemd:
cat /sys/fs/cgroup/system.slice/docker-<ID>.scope/memory.max
# driver cgroupfs:
cat /sys/fs/cgroup/docker/<ID>/memory.max
```

Como verificar

O container com 64 MB termina com código 137 e `OOMKilled` igual a `true`, e o valor lido em `memory.max` corresponde ao limite pedido na linha de comando.

Para pensar

O container foi encerrado por um erro da aplicação ou por uma decisão do kernel? Onde essa diferença aparece para quem está investigando um incidente?

Exercício 9: Limitando CPU e número de processos

Objetivo

Perceber que o limite de CPU não encerra o processo, apenas o desacelera, e conter a criação de processos.

Enunciado

1. Meça o tempo de um laço de repetição executado sem limite de CPU, usando o comando `time` do próprio terminal.
2. Execute o mesmo laço com `--cpus=0.25` e compare os tempos.
3. Enquanto o laço executa, acompanhe o consumo com `docker stats` em outro terminal.
4. Suba um container com `--pids-limit=5` e tente iniciar mais processos do que o limite permite.
5. Registre a mensagem de erro obtida e o momento em que ela aparece.

Comandos de apoio

```
# sem limite de CPU
time docker run --rm alpine \
  sh -c 'i=0; while [ $i -lt 3000000 ]; do i=$((i+1)); done'
# com um quarto de nucleo
time docker run --rm --cpus=0.25 alpine \
  sh -c 'i=0; while [ $i -lt 3000000 ]; do i=$((i+1)); done'
docker stats
docker run --rm --pids-limit=5 alpine \
  sh -c 'for i in 1 2 3 4 5 6 7 8 9 10; do sleep 5 & done; wait'
```

Como verificar

O tempo do laço limitado é visivelmente maior, sem erro de execução, e o container com `pids-limit` falha ao criar processos além do limite.

Para pensar

Um serviço lento e um serviço que morre exigem investigações diferentes. Qual limite produz cada um desses sintomas?

Exercício 10: Inspeção do host e limpeza do ambiente

Objetivo

Confirmar que o container é um processo do host e devolver a máquina ao estado inicial.

Enunciado

1. Com um container em execução, descubra o PID desse processo no host.
2. Confirme que o processo aparece na listagem de processos do host e liste os namespaces associados a ele.
3. Liste o espaço ocupado por imagens, containers e volumes.
4. Pare e remova todos os containers criados nesta lista.
5. Remova as imagens que não serão mais utilizadas e verifique o espaço liberado.

Comandos de apoio

```
docker inspect -f '{{.State.Pid}}' web
ps -p <PID> -o pid,ppid,cmd
sudo ls -l /proc/<PID>/ns
docker system df
docker ps -aq | xargs -r docker rm -f
docker image rm python:3.12-alpine python:3.12-slim
docker system df
```

Como verificar

O processo do container é localizado no host, e ao final `docker ps -a` não lista containers desta atividade e o `docker system df` mostra a redução de espaço.

Para pensar

Se o container é apenas um processo do host, o que impede que ele enxergue e interfira nos demais processos da máquina?

Comandos de referência

Comando	Para que serve
<code>docker run</code>	Cria e inicia um container novo a partir de uma imagem
<code>docker start / stop</code>	Reinicia ou para um container que já existe
<code>docker ps / ps -a</code>	Lista os containers em execução ou todos eles
<code>docker logs -f</code>	Mostra e acompanha a saída de um container
<code>docker exec -it</code>	Executa um comando dentro de um container em execução
<code>docker images</code>	Lista as imagens disponíveis na máquina
<code>docker pull</code>	Baixa uma imagem do registry
<code>docker image history</code>	Mostra as camadas que compõem uma imagem
<code>docker inspect</code>	Exibe os metadados do container ou da imagem em JSON
<code>docker stats</code>	Acompanha o consumo de recursos em tempo real
<code>docker port</code>	Mostra o mapeamento de portas do container
<code>docker system df / prune</code>	Mostra o espaço ocupado e remove o que não está em uso

Erros comuns e o que eles indicam

Mensagem	O que ela indica
<code>port is already allocated</code>	A porta do host já está publicada por outro container. Escolha outra porta ou remova o container anterior.
<code>exit code 137</code>	O processo foi encerrado por falta de memória, normalmente por causa do limite definido em <code>--memory</code> .
<code>name is already in use</code>	Já existe um container com esse nome, mesmo que parado. Use <code>docker rm</code> ou escolha outro nome.
<code>permission denied ao usar docker</code>	O usuário não está no grupo docker. Use <code>sudo</code> ou adicione o usuário ao grupo e reinicie a sessão.
<code>Cannot connect to the Docker daemon</code>	O serviço do engine não está em execução na máquina.

Fechamento

Ao terminar a lista, você deve ser capaz de explicar, com suas palavras, o que o Docker faz quando recebe um comando `docker run`: qual imagem é usada, o que os namespaces recortam da visão do processo, quais limites os cgroups aplicam e onde fica a camada de escrita do container. Se algum desses pontos ainda estiver aberto, retome o exercício correspondente antes do próximo encontro.